

Lld Questions

Generated on February 05, 2026 at 01:29 PM | 49 entries

Q1. Design a Parking Lot System

[MEDIUM] | *Category: Object-Oriented Design*

Answer: To design a Parking Lot system, we use an Object-Oriented approach. The core classes are `ParkingLot`, `ParkingFloor`, `ParkingSpot`, and `Vehicle`. The `ParkingLot` acts as a Singleton, managing multiple floors. Each `ParkingFloor` manages spots. We use an Enum for `SpotType` (Compact, Large, Handicapped) and a Strategy Pattern for `ParkingFeeCalculation` (e.g., Hourly vs Flat). Concurrency is handled using locks on the `assignSpot` method to prevent two vehicles from booking the same spot simultaneously.

Q2. Design a Rate Limiter

[HARD] | *Category: System Design*

Answer: A Rate Limiter restricts the number of requests a user can make in a given timeframe. The most common low-level implementation uses the **Token Bucket Algorithm**. We define a `Bucket` class containing `currentTokens`, `maxCapacity`, and `lastRefillTime`. When a request arrives, we calculate how many tokens to add based on the time elapsed since the last request (Lazy Refill). If `tokens > 0`, we decrement one and allow the request. If `tokens == 0`, we deny it. For thread safety, methods modifying the bucket must be `synchronized`.

Q3. Design a Tic-Tac-Toe Game

[EASY] | *Category: Game Design*

Answer: We model this with a `Board` class (NxN grid), a `Player` class, and a `Game` controller. The `Board` holds a 2D array of pieces. The critical logic is the `checkWin()` algorithm. A naive approach scans the board ($O(N^2)$), but an optimized approach uses arrays (`rows[]`, `cols[]`, `diag`, `antiDiag`) to track counts. If Player 1 moves at row X, we increment `rows[X]`. If `rows[X] == N`, they win. This makes the win check $O(1)$.

Q4. Design an Elevator System

[MEDIUM] | *Category: Object-Oriented Design*

Answer: This requires a `Controller` that manages multiple `ElevatorCars`. We use the **State Pattern** for the car (Idle, MovingUp, MovingDown). The dispatching logic typically uses the **SCAN (LOOK) Algorithm**: the elevator continues in the current direction servicing requests until there are no more requests in that direction, then it switches. We use a `PriorityQueue` inside the controller to manage floor requests efficiently based on the current direction.

Q5. Design an LRU Cache

[MEDIUM] | *Category: Data Structures*

Answer: An LRU (Least Recently Used) Cache evicts the least recently accessed item when capacity is reached. The standard LLD implementation uses a **HashMap** combined with a **Doubly Linked List**. The HashMap provides O(1) access to data. The Doubly Linked List maintains the order of usage. When an item is accessed or added, it is moved to the head of the list. When capacity is full, the item at the tail is removed from both the list and the map.

Q6. Design a Library Management System

[EASY] | *Category: Object-Oriented Design*

Answer: This system manages `Book`, `Member`, `Lending`, and `Fine`. We use a `Librarian` actor (Admin) and `Member` actor. We implement a `Search` interface to find books by Title, Author, or Subject. The core complexity handles the business rules: A member can only checkout `MAX_BOOKS`. If a book is returned after the due date, a fine is calculated using a `FineCalculator` strategy (e.g., \$1 per day).

Q7. Design a Stack Overflow Clone

[MEDIUM] | *Category: System Design*

Answer: This system involves `Users`, `Questions`, `Answers`, `Comments`, and `Tags`. The key feature is the **Reputation/Badge System** and **Voting**. We typically use the **Observer Pattern** so that when an answer gets an upvote, the `User` entity is notified to update their reputation score. Questions also need a flexible tagging system, often modeled as a Many-to-Many relationship in the DB.

Q8. Design a Logging Framework

[MEDIUM] | *Category: Design Patterns*

Answer: A Logging Framework like Log4j uses the **Chain of Responsibility** pattern. We have `Logger` objects with different levels (INFO, DEBUG, ERROR). When a log message arrives, it flows through the chain. If the logger's level matches the message, it processes it. We also use 'Appenders' (ConsoleAppender, FileAppender) to decide *where* to write. To prevent blocking the main thread, the actual writing should be asynchronous using a queue.

Q9. Design a Meeting Scheduler

[MEDIUM] | *Category: Object-Oriented Design*

Answer: The Scheduler manages `Rooms` and `Meetings`. A Room has a capacity. The core logic is `findMeetingSlot(users, duration)`. This checks the calendars of all invitees and finds a common free time interval. We must handle concurrency: two people cannot book the same room for the same slot. We use a global `Calendar` object or individual `UserCalendar` objects.

Q10. Design an ATM System

[MEDIUM] | *Category: Design Patterns*

Answer: The ATM system is a classic example of the **State Design Pattern**. The machine transitions through various states: `ReadyState`, `EnterPinState`, `SelectTransactionState`, `DepositState`, and `WithdrawCashState`. The core class `ATM` acts as the Context. We also use the **Chain of Responsibility** pattern for the cash dispenser (e.g., \$100 handler -> \$50 handler -> \$20 handler). Security is paramount; the `CardReader` class validates the card, and the `BankService` interface handles the actual transaction verification with the bank's backend. Singleton pattern is used for the ATM instance itself.

Q11. Design a Movie Ticket Booking System

[HARD] | *Category: System Design*

Answer: This system requires handling `Cinemas`, `Halls`, `Shows`, `Seats`, and `Bookings`. The most critical LLD challenge is **Concurrency** to prevent double-booking. When a user selects a seat, we use a database transaction with a 'Select for Update' or a distributed lock (e.g., Redis) to hold the seat for 10 minutes. Classes include `Cinema` (aggregates Halls), `Show` (aggregates Seats and Movie), and `Booking`. We use the **Strategy Pattern** for pricing (Premium vs. Standard seats, Afternoon vs. Evening pricing).

Q12. Design Snake and Ladder Game

[EASY] | *Category: Game Design*

Answer: The game consists of a `Board` (100 cells), `Dice`, `Players`, and entities for `Snake` and `Ladder`. The `Board` can be modeled as a HashMap or Array where the index is the cell number. A `Snake` is a jump from `start` to `end` (where start > end), and a `Ladder` is `start` to `end` (where end > start). The `Game` class runs the loop. The Dice is a utility class generating random 1-6. We can use the **Strategy Pattern** for the Dice if we want to support crooked dice or multiple dice. The design should allow easy adding of new 'Jumps' (like a portal).

Q13. Design Splitwise (Expense Sharing)

[HARD] | *Category: Algorithms*

Answer: Splitwise simplifies debts among a group. The key entities are `User`, `Group`, `Expense`, and `Split`. The `Expense` class can have different types: `EqualSplit`, `ExactSplit`, `PercentSplit`. This is a perfect use case for the **Strategy Pattern** or **Factory Pattern**. The algorithmic core is **Debt Simplification**. If A owes B \$10, and B owes C \$10, the system simplifies this to A owes C \$10. This is solved using a Min-Cash-Flow algorithm or graph simplification.

Q14. Design an Airline Management System

[HARD] | *Category: Object-Oriented Design*

Answer: This system manages `Flights`, `Airports`, `Aircrafts`, and `Schedules`. A crucial distinction is between a `Flight` (generic route, e.g., BA123) and a `FlightInstance` (specific date, e.g., BA123 on Dec 25th). We need an `Itinerary` class for multi-leg bookings. The system must handle complex search queries. We use the **Observer Pattern** to notify customers of flight delays or gate changes. Concurrency control is similar to the Movie Ticket system but strictly stricter, often using **Two-Phase Commit** if distributed across multiple airline partners.

Q15. Design a Restaurant Management System

[MEDIUM] | *Category: Object-Oriented Design*

Answer: This system handles `Tables`, `Orders`, `Meals`, and Employees (`Waiter`, `Chef`). We use the **Command Pattern** for Orders. An Order is a request wrapped as an object, passed from Waiter to Chef. We use the **State Pattern** for Table status (Empty, Occupied, Reserved) and Order status (Received, Preparing, Completed, Served). The **Observer Pattern** is useful: when a Chef marks an order as 'Ready', the assigned Waiter is notified.

Q16. Design a Chat System (WhatsApp/Telegram)

[HARD] | *Category: System Design*

Answer: Designing a Chat App involves `Users`, `Conversations` (1-on-1 and Group), and `Messages`. The core is real-time communication. In LLD, we model the `Connection` using **WebSockets**. The `Message` status (Sent, Delivered, Read) is a state machine. For Group chats, we use the **Composite Pattern** (treating a Group similar to a User for sending logic) or a `GroupConversation` class that holds a list of `Participants`. We need a `Gateway` service to route messages to the correct user session.

Q17. Design a Notification System

[MEDIUM] | *Category: Design Patterns*

Answer: This system decouples the trigger of a notification from the delivery method. We use the **Observer Pattern** or **Pub-Sub**. The core class is `NotificationService`. It uses a `NotificationFactory` to create specific notifications (Email, SMS, Push). We implement the **Adapter Pattern** to integrate with third-party providers (e.g., Twilio for SMS, SendGrid for Email). This ensures that if we switch providers, our core logic doesn't change. We also need a `TemplateEngine` to inject variables (e.g., "Hello {name}") into messages.

Q18. Design a File System

[MEDIUM] | *Category: Object-Oriented Design*

Answer: Modeling a File System (like Unix or Windows) relies heavily on the **Composite Design Pattern**. We have an abstract class `FileSystemEntry`. Two concrete classes extend it: `File` and `Directory`. - A `File` is a leaf node. - A `Directory` is a composite node that contains a list of `FileSystemEntry` (which can be Files or other Directories). This allows us to treat individual files and directories uniformly (e.g., `getSize()`, `delete()`). We also need to handle Permissions (`Read`, `Write`, `Execute`) often using a **Decorator Pattern** or simple bitmasks.

Q19. Design a HashMap Implementation

[HARD] | *Category: Data Structures*

Answer: Designing a HashMap tests knowledge of Data Structures. The core is an Array of `Nodes` (Buckets). Each `Node` is a Linked List (or a Red-Black Tree in Java 8+) to handle collisions. Key functions: - `put(K, V)`: Calculate `hash(K) % size`. Go to that index. If empty, insert. If occupied, traverse list to update or append. - `get(K)`: Calculate hash. Traverse list to find Key. We must handle **Resizing**. When `size / capacity > loadFactor` (usually 0.75), we create a new array of double size and **Rehash** all existing entries into the new array.

Q20. Design a Pub-Sub System (Message Queue)

[HARD] | *Category: System Design*

Answer: A Pub-Sub system (like Kafka or RabbitMQ) decouples Producers from Consumers. Core components: `Topic`, `Message`, `Broker`, `Producer`, `Consumer`. We use the **Observer Pattern**. Consumers 'subscribe' to a Topic. When a Producer 'publishes' to a Topic, the Broker notifies all subscribers. To handle scale, we introduce **Partitions** within a Topic. A partition is an ordered, immutable sequence of messages. This allows parallel consumption. The Broker must persist messages to disk (WAL - Write Ahead Log) for durability.

Q21. Design a Coffee Machine

[EASY] | *Category: Design Patterns*

Answer: This is a classic example of the **Decorator Pattern**. We have a base class `Beverage` (cost, description). Concrete classes are `Espresso`, `Decaf`. We have `CondimentDecorator` extending `Beverage`. Concrete decorators: `Milk`, `Soy`, `Mocha`, `Whip`. This allows flexible combinations like 'Double Mocha Soy Latte' without creating thousands of subclasses. We wrap the object: `new Mocha(new Milk(new Espresso()))`. The `cost()` method calls the wrapped object's cost and adds its own.

Q22. Design a Car Rental System

[MEDIUM] | *Category: Object-Oriented Design*

Answer: The system manages `VehicleInventory`, `Reservation`, `User`, and `StoreLocation`. We use the **Factory Pattern** to handle different vehicle types (SUV, Sedan, Bike). We use the **Strategy Pattern** for pricing (DailyRate, WeeklyRate, MileageRate). A key LLD feature is the **Inventory Management**. We need to track vehicle status: `Available`, `Booked`, `InUse`, `Maintenance`. A 'Barcode Scanner' interface might be needed for check-out/check-in processing.

Q23. Design Cricinfo (Cricket Scoreboard)

[MEDIUM] | *Category: Object-Oriented Design*

Answer: This system requires real-time updates. We strictly use the ****Observer Pattern****. The ``Match`` object is the Subject. ``ScoreBoard``, ``MobileApp``, and ``WebWidget`` are Observers. The data model is hierarchical: ``Series`` -> ``Match`` -> ``Innings`` -> ``Over`` -> ``Ball``. A ``Ball`` is the atomic unit containing data (runs, wicket, extra). We use the ****Strategy Pattern**** for commentary generation or 'Player of the Match' calculation algorithms.

Q24. Design a Digital Wallet

[HARD] | *Category: System Design*

Answer: A Digital Wallet (like PayPal/Paytm) manages ``UserAccounts``, ``BankIntegration``, and ``Transactions``. Concurrency and Data Integrity are vital. We use a ****Double Entry Ledger**** system pattern for the database design: every transaction has a source and destination. Money is never just 'updated'; rows are inserted. We use the ****Strategy Pattern**** for adding money (`CreditCardStrategy`, `UPIStrategy`, `NetBankingStrategy`). Security uses ``Tokenization`` for card details.

Q25. Design an Online Shopping Cart

[MEDIUM] | *Category: Object-Oriented Design*

Answer: We need to model ``Cart``, ``CartItem``, ``Product``, and ``User``. The complex logic lies in **Price Calculation** (Discounts, Tax, Coupons). This is a perfect fit for the **Decorator Pattern** (wrapping the cart with a tax wrapper) or **Strategy Pattern** (Apply ``PercentageDiscount`` vs ``FixedAmountDiscount``). The Cart state management (Active, Merged, Abandoned) is important. When a guest logs in, their 'Guest Cart' must merge with their 'User Cart'.

Q26. Design a Task Scheduler

[HARD] | *Category: System Design*

Answer: A Task Scheduler runs jobs at specific times or intervals. We use a **PriorityQueue** (Min-Heap) to store tasks ordered by ``executionTime``. A background thread (Poller) checks ``queue.peek()``. If ``head.time <= now``, it extracts and executes. If ``head.time > now``, it sleeps for ``head.time - now``. We need a **ThreadPool** (Worker Threads) to execute the tasks so the Poller isn't blocked. This implements the **Producer-Consumer** pattern.

Q27. Design a Unix File Search API

[MEDIUM] | *Category: Design Patterns*

Answer: We need to design a flexible search API (like the ``find`` command). We use the **Specification Pattern** (or Filter Pattern). We define an interface ``FileFilter`` with method ``isValid(File file)``. Concrete implementations: ``NameFilter``, ``SizeFilter``, ``ExtensionFilter``. To combine filters (AND, OR), we use **Combinators**: ``AndFilter`` takes two filters and returns true only if both are true. This allows complex queries like: ``Name="*.txt" AND Size>5MB``.

Q28. Design a Jigsaw Puzzle Solver

[MEDIUM] | *Category: Algorithms*

Answer: This models a physical puzzle. Entities: ``Piece``, ``Edge``, ``Side``. A ``Piece`` has 4 sides. Each side has a shape (Inward, Outward, Flat). The core logic is ``fitsWith(Piece other)``. Two pieces fit if one has an Outward tab and the other has a matching Inward hole (and colors/patterns match, but usually in LLD we check shape). We categorize pieces: - **Corners** (2 Flat sides). - **Borders** (1 Flat side). - **Middle** (0 Flat sides). Solving starts by identifying corners and borders.

Q29. Design a Locker Service (Amazon Locker)

[MEDIUM] | *Category: Object-Oriented Design*

Answer: The system manages `Lockers` of different sizes (S, M, L, XL) at various `Locations`. Flow: 1. Courier delivers package. System finds free locker of size $\geq$ Package. 2. System generates `Code` and sends to User. 3. User enters Code. Locker opens. We use the **Strategy Pattern** for locker allocation (e.g., First Fit vs Best Fit). We need to handle **Expiration**: if user doesn't pick up in 3 days, generate 'Return' request.

Q30. Design a Text Editor (Undo/Redo)

[HARD] | *Category: Design Patterns*

Answer: The core challenge in a Text Editor is managing the document state and supporting Undo/Redo operations. This is the textbook use case for the **Memento Pattern** (saving state snapshots) or the **Command Pattern** (storing actions as objects). We typically use a **Gap Buffer** or **Rope** data structure for efficient text insertion and deletion, as standard Strings are immutable and slow for large edits. For Undo/Redo, we maintain two Stacks: `UndoStack` and `RedoStack`. When a user types, we push the action (e.g., `Insert 'a' at pos 5`) to the UndoStack. If they hit Undo, we pop the action, reverse it, and push it to the RedoStack.

Q31. Design an Online Stock Brokerage

[HARD] | *Category: System Design*

Answer: This system handles `Users`, `Portfolios`, `Stocks`, and `Orders`. The critical component is the **Order Matching System** (though often an external service, in LLD we design the order creation and state). We use an `Order` class with types: `MarketOrder` (execute immediately) and `LimitOrder` (execute at price X). Concurrency is vital; we cannot sell stock the user doesn't own. We use **Optimistic Locking** on the `Portfolio` table. The `Watchlist` feature uses the **Observer Pattern** to update real-time prices.

Q32. Design a Chess Game

[HARD] | *Category: Game Design*

Answer: Chess is complex due to the movement rules. We use an abstract `Piece` class with a method `canMove(Board b, Start s, End e)`. Concrete classes (`King`, `Rook`, `Knight`) implement specific logic. The `Board` is an 8x8 grid of `Spot` objects. A `Spot` can hold a Piece or be null. The `Game` class manages the turn-based loop and checks for `Checkmate` or `Stalemate`. We often use the **Singleton Pattern** for the Game settings and **Command Pattern** for moves to allow recording the game history (PGN format).

Q33. Design a URL Shortener (LLD Focus)

[MEDIUM] | *Category: Algorithms*

Answer: While often a System Design question, the LLD focuses on the **Base62 Encoding Algorithm** and the Database Schema. We need a `ShorteningService` with a method `shorten(longURL)`. The algorithm maps an Integer ID (database auto-increment) to a Base62 string (a-z, A-Z, 0-9). - ID 1 -> "a" - ID 100 -> "dF" - ID 10000 -> "z9A" We also need a `RetrievalService` that takes a short code, converts it back to the ID (`base62Decode`), and looks up the DB.

Q34. Design a Social Network (LinkedIn)

[MEDIUM] | *Category: Object-Oriented Design*

Answer: We model `Member`, `Connection`, `Post`, and `Profile`. The core data structure is a **Graph**. Each Member is a Node; a Connection is an Edge. - `getConnections(Member m)` returns adjacent nodes. - `getSecondDegree(Member m)` performs a BFS (Breadth-First Search) to find friends-of-friends. We use the **Observer Pattern** for the News Feed: when a connection posts, it is pushed to the user's feed (or pulled on refresh).

Q35. Design a Web Crawler

[HARD] | *Category: System Design*

Answer: The Crawler fetches pages, extracts links, and visits them. LLD components: `URLFrontier`` (Queue), `HTMLParser``, `Fetcher``, and `Datastore``. We must handle **Politeness** (not hitting a server too fast). We use a `Map<Hostname, Queue>` where each hostname has its own delay timer. To avoid infinite loops, we need a `VisitedSet``. Since URLs are long, storing them all in memory is hard. In LLD, we discuss using a **Bloom Filter** for space-efficient membership testing.

Q36. Design a Leaderboard

[MEDIUM] | *Category: Data Structures*

Answer: A Leaderboard shows top players. The key requirement is efficient `updateScore()` and `getTopK()`. Standard SQL sorting is slow ($O(N \log N)$). The optimal LLD data structure is a **Skip List** or a **Balanced BST** (like Red-Black Tree), or utilizing **Redis Sorted Sets** (ZSET) if external tools are allowed. In pure code (Java/C++), we use a `Map<Player, Score>` for $O(1)$ lookups and a `TreeMap<Score, List<Player>>` (sorted map) to maintain order. Updating a score involves removing from the old position in `TreeMap` and inserting at the new.

Q37. Design a Hotel Management System

[MEDIUM] | *Category: Object-Oriented Design*

Answer: Similar to Airline/Movie, but with distinct features like `RoomType` (Suite, King, Twin) and `HousekeepingStatus`. We use the **Decorator Pattern** for amenities (Extra Bed, Breakfast, Spa Access). We use the **State Pattern** for the Room (Available, Booked, Occupied, Cleaning, Repair). Key search logic: `findRooms(startDate, endDate, type)`. This involves checking the `Reservation` table for overlaps.

Q38. Design a Cab Booking System (Uber/Lyft)

[HARD] | *Category: System Design*

Answer: We focus on `Rider`, `Driver`, `Ride`, and the `GeoLocation`. The core logic is **Driver Matching**. We need a `GeoSpatialIndex` (like a QuadTree) to efficiently query "Drivers within 2km". We use the **State Pattern** for the Ride (Requested, Matched, Started, Ended, Cancelled). We use the **Observer Pattern** to track driver location updates on the rider's screen.

Q39. Design a Payment Gateway

[HARD] | *Category: Design Patterns*

Answer: A Payment Gateway acts as an adapter between a Merchant and a Bank. We use the **Adapter Pattern** heavily. - `IPaymentProvider` interface (`pay`, `refund`). - `StripeAdapter`, `PayPalAdapter` implement this interface. We use the **Strategy Pattern** for Routing (e.g., if amount < \$10 use PayPal, if > \$1000 use WireTransfer). **Resiliency** is key: we need a `CircuitBreaker` if a bank is down.

Q40. Design a Traffic Signal Control System

[EASY] | *Category: Object-Oriented Design*

Answer: This controls lights at an intersection. We use the **State Pattern**: `RedState`, `YellowState`, `GreenState`. The `TrafficController` manages the duration. It can be: 1. **Fixed Timing**: Simple Timer. 2. **Dynamic**: Uses sensors (`CarDetectedObserver`). If no cars on Side A, switch Side A to Red immediately. We need a `Mediator` if controlling multiple intersections to synchronize 'Green Waves'.

Q41. Design a Music Player (Spotify)

[MEDIUM] | *Category: Object-Oriented Design*

Answer: We model `Song`, `Playlist`, `Album`, `Artist`, and `Player`. The `Player` is a **Singleton** (only one song plays at a time). It maintains a `Queue` of songs. We use the **State Pattern** for the Player: `Playing`, `Paused`, `Stopped`. Handling 'Shuffle' involves Algorithm logic (Fisher-Yates Shuffle) on the queue. Handling 'Repeat' involves circular queue logic.

Q42. Design a Distributed ID Generator (Snowflake)

[HARD] | *Category: System Design*

Answer: We need to generate unique, roughly sortable 64-bit IDs without a central DB. **Snowflake Algorithm (Twitter):** - 1 bit: Sign bit (unused). - 41 bits: Timestamp (milliseconds). - 10 bits: Machine ID (Data Center ID + Worker ID). - 12 bits: Sequence Number (for IDs generated in the same millisecond). The class `SnowflakeGenerator` creates IDs by bitwise shifting these components. It requires handling **Clock Move Backwards** scenarios (NTP drift) by throwing an error or waiting.

Q43. Design a Content Management System (CMS)

[MEDIUM] | *Category: Object-Oriented Design*

Answer: A CMS (like WordPress) manages `Pages`, `Posts`, and `Media`. We use the **Composite Pattern** for the Page structure. A `Page` consists of `Components` (Header, Body, Footer). The `Body` contains `Widgets` (TextWidget, ImageWidget). We use the **Bridge Pattern** to separate the `Content` (what to render) from the `Renderer` (how to render: HTML, AMP, Mobile App JSON).

Q44. Design a Calendar Application

[HARD] | *Category: Object-Oriented Design*

Answer: The complexity lies in **Recurring Events** (e.g., "Every Monday", "Every 2nd Friday"). We store events as: 1. `SingleEvent`: Fixed start/end. 2. `RecurringEvent`: Start, Pattern (CRON or RRule), Duration. When querying "Get events for June", we expand the Recurring Events dynamically. We don't store infinite future events in the DB. We use the **Builder Pattern** for creating complex events.

Q45. Design a Blackjack Game

[EASY] | *Category: Game Design*

Answer: Entities: `Deck`, `Card`, `Hand`, `Player`, `Dealer`. Logic: - `Deck` is a collection of 52 cards. `shuffle()` and `deal()`. - `Hand` calculates score. Aces are tricky (1 or 11). Logic: `value = sum. if (value > 21 && hasAce) value -= 10`. The `Game` follows strict steps: Bets -> Deal -> Player Turn (Hit/Stand) -> Dealer Turn (must hit until ≥ 17) -> Payout.

Q46. Design a Calorie Tracker

[MEDIUM] | *Category: Object-Oriented Design*

Answer: System to log `Food`, `Exercise`, and `Weight`. We use the **Factory Pattern** for Food entries: `BarcodeFood`, `CustomFood`, `Recipe`. The `Recipe` is a **Composite**: it contains other Foods. Calculation: `NetCalories = FoodIn - ExerciseOut`. We need a `Goal` class (Lose 1lb/week) which calculates the `DailyBudget`.

Q47. Design an Expression Evaluator

[HARD] | *Category: Algorithms*

Answer: This solves strings like $(5 + 3) * 2$. We use the **Composite Pattern**. - `Expression` (Interface). `evaluate()`. - `NumberExpression` (Leaf). Returns value. - `OperationExpression` (Node). Has left and right Expressions. Returns `left.eval() op right.eval()`. To parse the string, we use the **Shunting Yard Algorithm** (Stacks) to convert Infix to Postfix, or build the Expression Tree directly.

Q48. Design a Thread-Safe Singleton

[MEDIUM] | *Category: Design Patterns*

Answer: A Singleton ensures only one instance exists. Naive approach: ``static instance``. Lazy loading needs checks. Thread-Safe approach: ****Double-Checked Locking****. 1. Check if null. 2. ``synchronized`` block. 3. Check if null again. 4. Create. We must use the ``volatile`` keyword in Java/C++ to prevent instruction reordering issues. Alternatively, use ****Bill Pugh Singleton**** (Static Inner Class) or ****Enum Singleton**** (Simplest, prevents reflection attacks).

Q49. Design a Custom Iterator

[MEDIUM] | *Category: Design Patterns*

Answer: We want to iterate over a custom collection (e.g., a `SocialGraph` or `Playlist`) using a `foreach` loop. We implement the **Iterator Pattern**. Interface `Iterator`: `hasNext()`, `next()`. Interface `Iterable`: `iterator()`. For a `Playlist`, the `Iterator` maintains a cursor `currentTrackIndex`. For a `Graph` (BFS `Iterator`), the `Iterator` maintains a `Queue` of nodes to visit next. This allows lazy traversal of a massive graph.